main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int vertex;
    struct Node* next;
};
struct Graph {
    int numVertices;
    struct Node** adjLists;
};
struct Node* createNode(int v) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}
struct Graph* createGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct Node*));
    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
    }
    return graph;
}
void add_edge(struct Graph* graph, int v, int w) {
    struct Node* newNode = createNode(w);
    newNode->next = graph->adjLists[v];
    graph->adjLists[v] = newNode;
    newNode = createNode(v);
    newNode->next = graph->adjLists[w];
    graph->adjLists[w] = newNode;

```

Output

Clear

```
Edge added between 0 and 1
Edge added between 0 and 4
Edge added between 1 and 2
Edge added between 1 and 3
Edge added between 1 and 4

Graph Adjacency List Representation:
Vertex 0: 4 -> 1 -> NULL
Vertex 1: 4 -> 3 -> 2 -> 0 -> NULL
Vertex 2: 1 -> NULL
Vertex 3: 1 -> NULL
Vertex 4: 1 -> 0 -> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    graph->adjLists[v] = newNode;
    newNode = createNode(v);
    newNode->next = graph->adjLists[w];
    graph->adjLists[w] = newNode;

    printf("Edge added between %d and %d\n", v, w);
}
void printGraph(struct Graph* graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        struct Node* temp = graph->adjLists[i];
        printf("Vertex %d: ", i);
        while (temp) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

int main() {
    struct Graph* graph = createGraph(5);

    add_edge(graph, 0, 1);
    add_edge(graph, 0, 4);
    add_edge(graph, 1, 2);
    add_edge(graph, 1, 3);
    add_edge(graph, 1, 4);

    printf("\nGraph Adjacency List Representation:\n");
    printGraph(graph);

    return 0;
}
```

Output

Clear

```
Edge added between 0 and 1
Edge added between 0 and 4
Edge added between 1 and 2
Edge added between 1 and 3
Edge added between 1 and 4

Graph Adjacency List Representation:
Vertex 0: 4 -> 1 -> NULL
Vertex 1: 4 -> 3 -> 2 -> 0 -> NULL
Vertex 2: 1 -> NULL
Vertex 3: 1 -> NULL
Vertex 4: 1 -> 0 -> NULL


=== Code Execution Successful ===
```